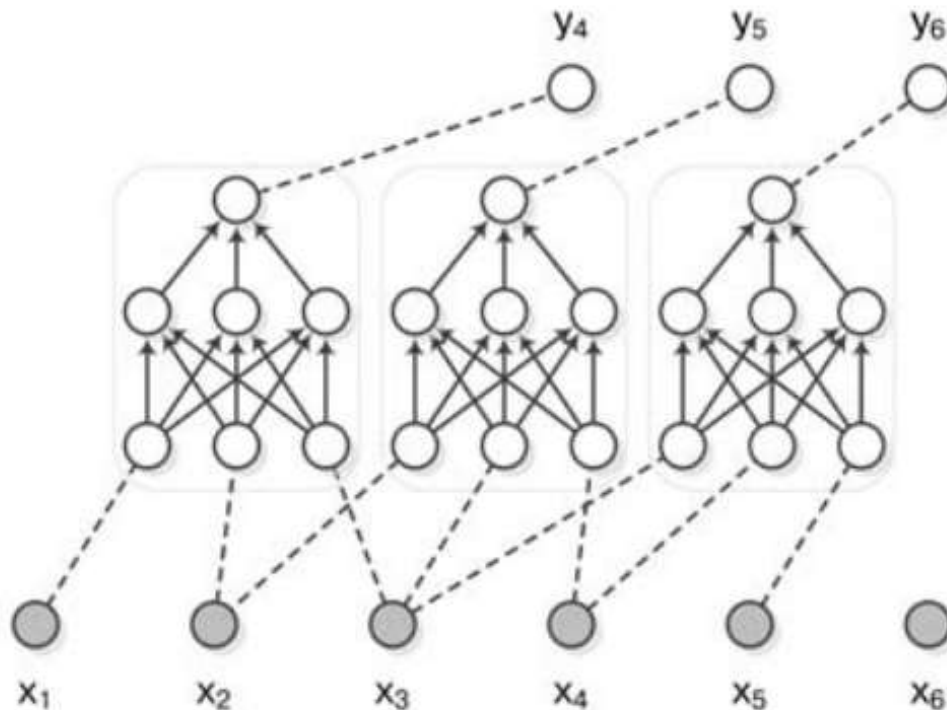


5.1 Рекуррентные нейронные сети: архитектура и принципы работы, основные задачи анализа последовательностей данных

Главной характеристикой нейронных сетей, с которыми мы познакомились к данному моменту является отсутствие памяти. Каждый вход обрабатывается ими независимо, без сохранения состояния между ними. Чтобы с помощью таких сетей обработать последовательность или временной ряд данных, необходимо передать в сеть всю последовательность целиком, преобразовав ее в единый пакет, или разбить на пакеты размерностью в соответствии с числом входов в нулевом слое и применять к последовательно подаваемым пакетам входа. Пример такой архитектуры показан на рисунке ниже. Здесь выходы получаются из трех предыдущих входов:



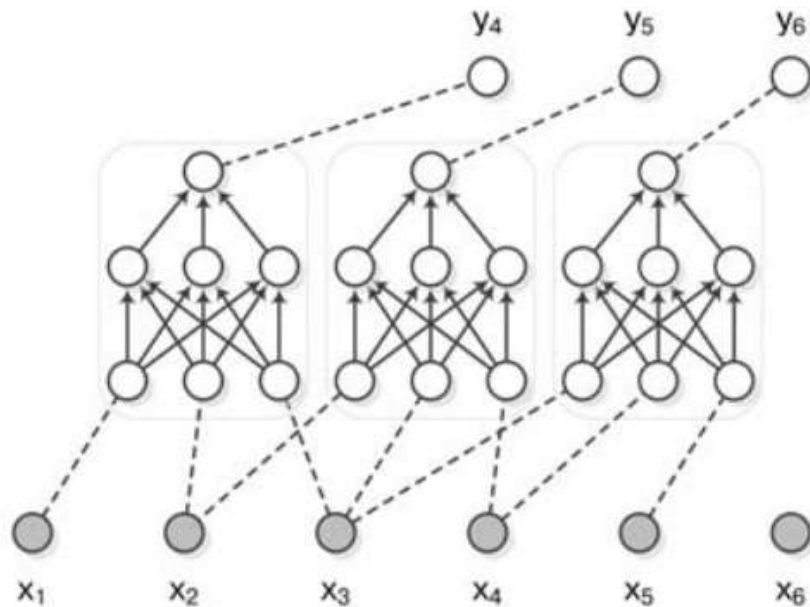
$$y_4 = f(x_1, x_2, x_3)$$

$$y_5 = f(x_2, x_3, x_4)$$

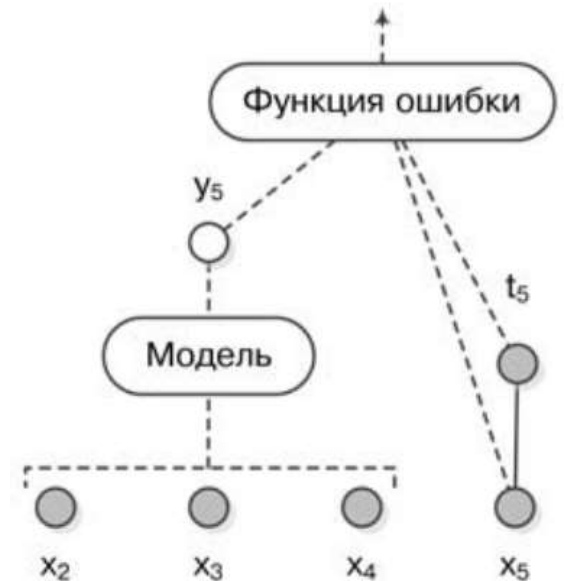
$$y_6 = f(x_3, x_4, x_5)$$

5.1 Рекуррентные нейронные сети: архитектура и принципы работы, основные задачи анализа последовательностей данных

Если задачей модели является предсказание следующего элемента (такая задача стоит, например, при порождении текстов или предсказании временных рядов), в качестве функции ошибки можно взять разницу между y_j и соответствующим x_j (например, процент угаданных слов).



$$\begin{aligned}y_4 &= f(x_1, x_2, x_3) \\y_5 &= f(x_2, x_3, x_4) \\y_6 &= f(x_3, x_4, x_5)\end{aligned}$$

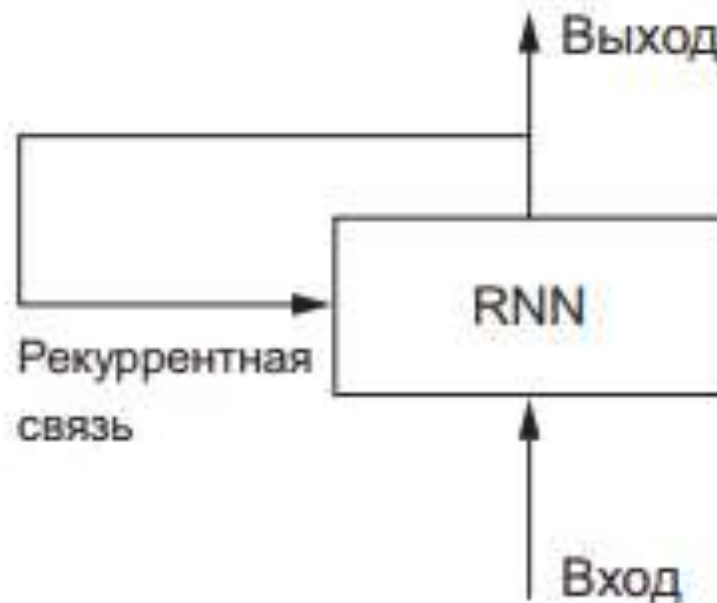


5.1 Рекуррентные нейронные сети: архитектура и принципы работы, основные задачи анализа последовательностей данных

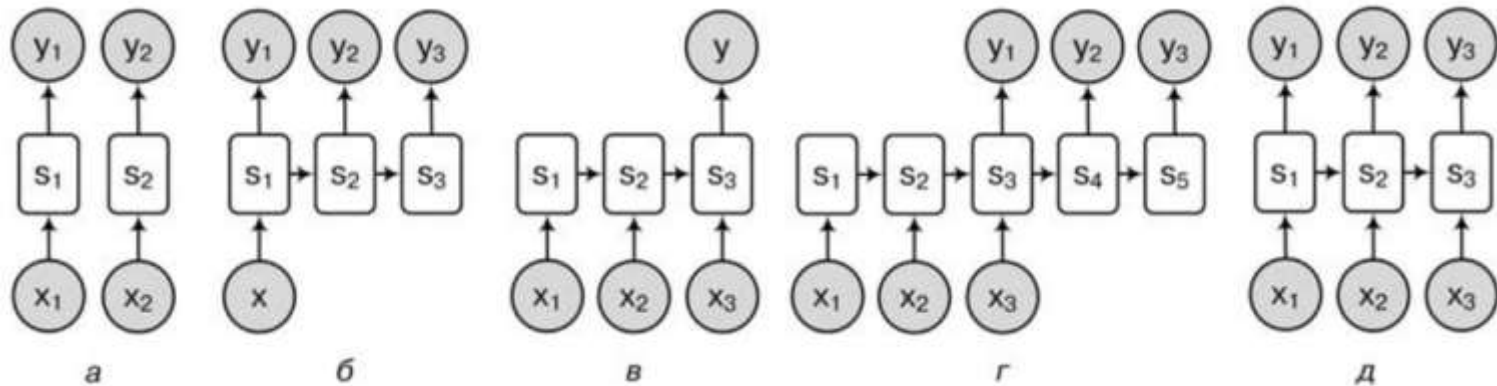
- С другой стороны, читая предложение, мы осмысливаем его слово за словом, быстро перескакивая глазами с одного на другое и запоминая предыдущие; это позволяет нам постепенно вникать в смысл, передаваемый предложением. Биологический интеллект воспринимает информацию последовательно, сохраняя внутреннюю модель обрабатываемого, основываясь на предыдущей информации и постоянно дополняя эту модель по мере поступления новой информации. Было бы хорошо, если бы нейронная сеть могла запоминать что-то из истории приходящих на вход данных, сохранять некое внутреннее состояние, которое можно было бы потом использовать для предсказания будущих элементов последовательности.

5.1 Рекуррентные нейронные сети: архитектура и принципы работы, основные задачи анализа последовательностей данных

- ❑ *Рекуррентная нейронная сеть* (РНС, Recurrent Neural Network, RNN) использует тот же принцип, хотя и в чрезвычайно упрощенном виде: она обрабатывает последовательность, перебирая ее элементы и сохраняя *состояние*, полученное при обработке предыдущих элементов. Фактически RNN — это разновидность нейронной сети, имеющей внутренний цикл или обратную связь.



5.1 Рекуррентные нейронные сети: архитектура и принципы работы, основные задачи анализа последовательностей данных

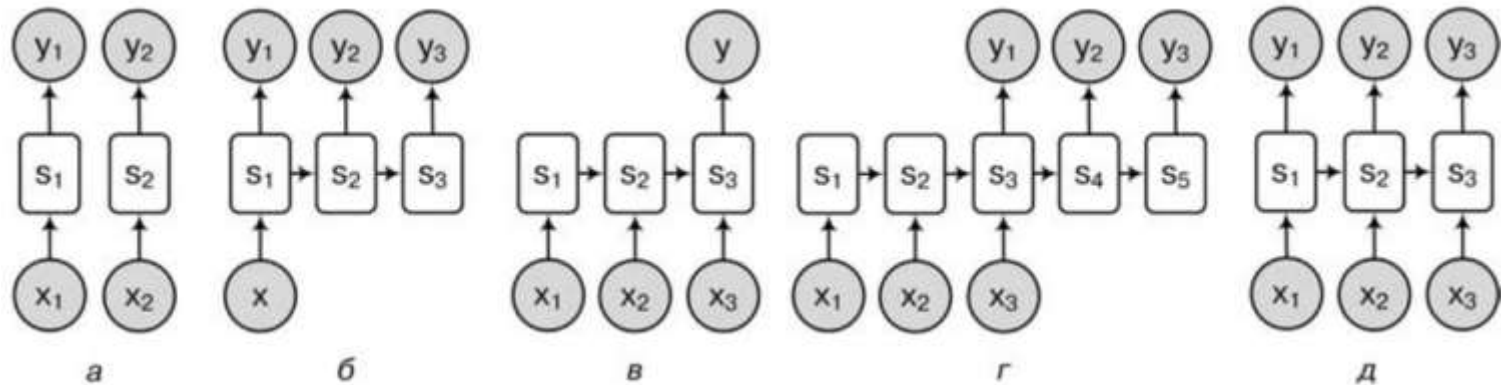


Задачи с последовательностями: а — один вход, один выход; б — один вход, последовательность выходов; в — последовательность входов, один выход; г — последовательность входов, затем последовательность выходов; д — синхронизированные последовательности входов и выходов

□ Основные типы задач машинного обучения, связанных с последовательностями, по характеру входов и выходов задачи можно классифицировать на следующие пять типов:

- один вход, один выход (one-to-one, рис. а); здесь последовательности как бы и нет, мы просто должны независимо обработать каждый элемент входов и получить соответствующий выход, никакие скрытые состояния никуда не передаются; однако заметим, что работа с последовательностью когда окна обрабатываются независимо, вполне укладывается в эту схему, просто входы получатся достаточно сильно зависимыми;
- один вход, последовательность выходов (one-to-many, рис. б); здесь мы должны «развернуть» вход, который сам по себе не имеет структуры последовательности, в последовательность выходов; например, аннотирование картинок представляет собой такую задачу: на входе картинка, на выходе текст (последовательность);

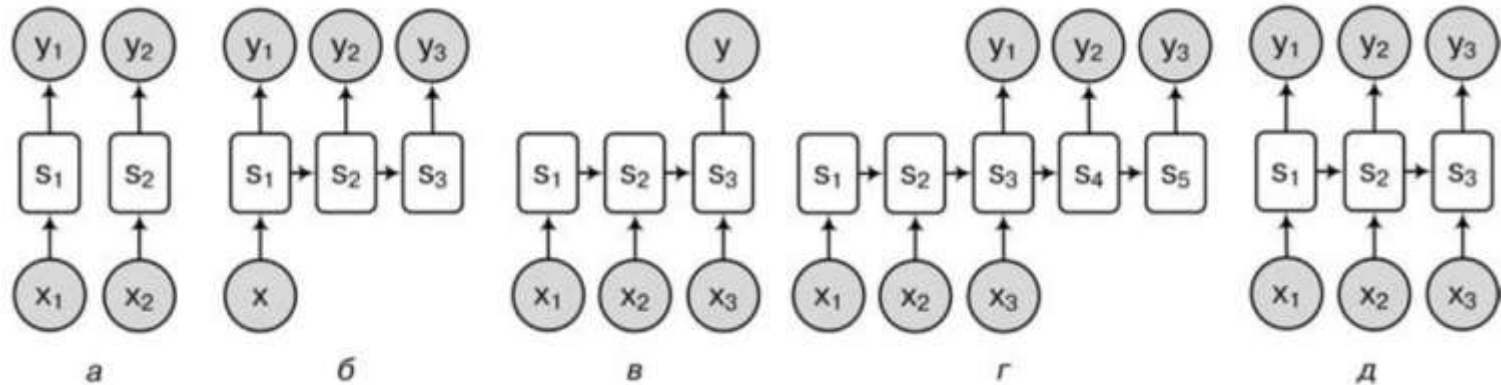
5.1 Рекуррентные нейронные сети: архитектура и принципы работы, основные задачи анализа последовательностей данных



Задачи с последовательностями: а — один вход, один выход; б — один вход, последовательность выходов; в — последовательность входов, один выход; г — последовательность входов, затем последовательность выходов; д — синхронизированные последовательности входов и выходов

- последовательность входов, один выход (many-to-one, рис. в); в этот тип задач укладываются любые задачи классификации последовательностей; например, анализ тональности (sentiment analysis): по данному тексту (то есть последовательности) выдать, положительно или отрицательно он окрашен; или же задачи предиктивного регрессионного анализа временных рядов, когда выходом является единичное прогнозное значение;
- последовательность входов, затем последовательность выходов (many-to-many, рис. г); здесь речь идет о том, чтобы «свернуть» входную последовательность, закодировать ее неким скрытым состоянием, а потом «развернуть» это скрытое состояние обратно в уже совершенно другую последовательность; например, по этой общей схеме работают системы машинного перевода (вход — предложение на одном языке, выход — на другом) и диалоговые системы (вход — реплика собеседника, выход — своя собственная реплика);

5.1 Рекуррентные нейронные сети: архитектура и принципы работы, основные задачи анализа последовательностей данных

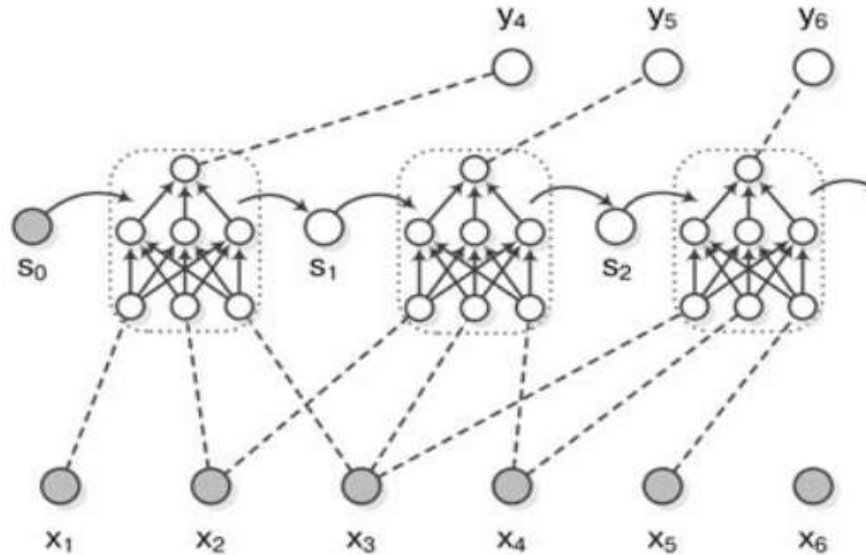


Задачи с последовательностями: а — один вход, один выход; б — один вход, последовательность выходов; в — последовательность входов, один выход; г — последовательность входов, затем последовательность выходов; д — синхронизированные последовательности входов и выходов

- синхронизированные последовательности входов и выходов (synchronized many-to-many, рис. д); а здесь нужно снабдить своей меткой каждый элемент последовательности, но в отличие от рис. а есть смысл также переносить на следующий временной шаг некое скрытое состояние; например, представьте, что нам нужно разметить видеопоток, в котором каждый последующий кадр, конечно, представляет собой самостоятельную картинку, но она обычно очень похожа на предыдущую и следующую.

5.1 Рекуррентные нейронные сети: архитектура и принципы работы, основные задачи анализа последовательностей данных

- ❑ Рекуррентная архитектура представляется очень удачной для обработки последовательностей, а последовательности — это и тексты, и видео, и музыка... но как же теперь считать градиенты?
- ❑ До сих пор у нас все графы вычислений были ациклическими (без циклов, то есть без «замкнутых кругов»), а теперь получается, что в графе сети одни сплошные циклы? Как можно вычислить функцию, которая принимает сама себя на вход?



- ❑ Вычисление, которое делает рекуррентная сеть, можно развернуть обратно до начала последовательности. Например, обозначая в примере на рисунке слайда 1 состояние сети после i -го тестового примера через $s_i = h(x_i, x_{i+1}, x_{i+2}, s_{i-1})$, получим
$$y_6 = f(x_3, x_4, x_5, s_2) = f(x_3, x_4, x_5, h(x_2, x_3, x_4, s_1)) =$$
$$= f(x_3, x_4, x_5, h(x_2, x_3, x_4, h(x_1, x_2, x_3, s_0))).$$

5.1 Рекуррентные нейронные сети: архитектура и принципы работы, основные задачи анализа последовательностей данных

- Здесь уже s_0 взять неоткуда, это будет начальное состояние сети. Параметры сети здесь спрятаны внутри функций f и h , то есть в этой формуле одни и те же веса встречаются много раз. Можно сказать, что на каждом шаге сеть создает несколько копий самой себя. Каждая из этих копий принимает на вход текущее окно в определенный момент времени (определенную часть последовательности) и значение, полученное из предыдущей копии, затем каким-то образом их комбинирует и передает получившийся результат в следующий элемент. Таким образом, на каждом шаге мы фактически обучаем глубокую нейронную сеть, в которой столько слоев, сколько элементов в последовательности мы уже видели. Основное ее отличие от обычной нейронной сети, которые мы рассматривали до сих пор, состоит в том, что веса на каждом слое одинаковые, все слои делят одни и те же переменные между собой (shared weights).

5.2 Рекуррентные нейронные сети: простые РНС

- Теперь можно и подсчитать градиент в развернутом виде. Для примера вычисления мы возьмем одну из самых базовых архитектур **«простой» рекуррентной сети**: будем использовать нейроны с сигмоидом в качестве функции активации и считать, что все переходы, кроме этого сигмоида, линейные, а на выходе мы решаем задачу классификации с помощью softmax-функции от выхода сети или просто применяем ту или иную функцию активации h .

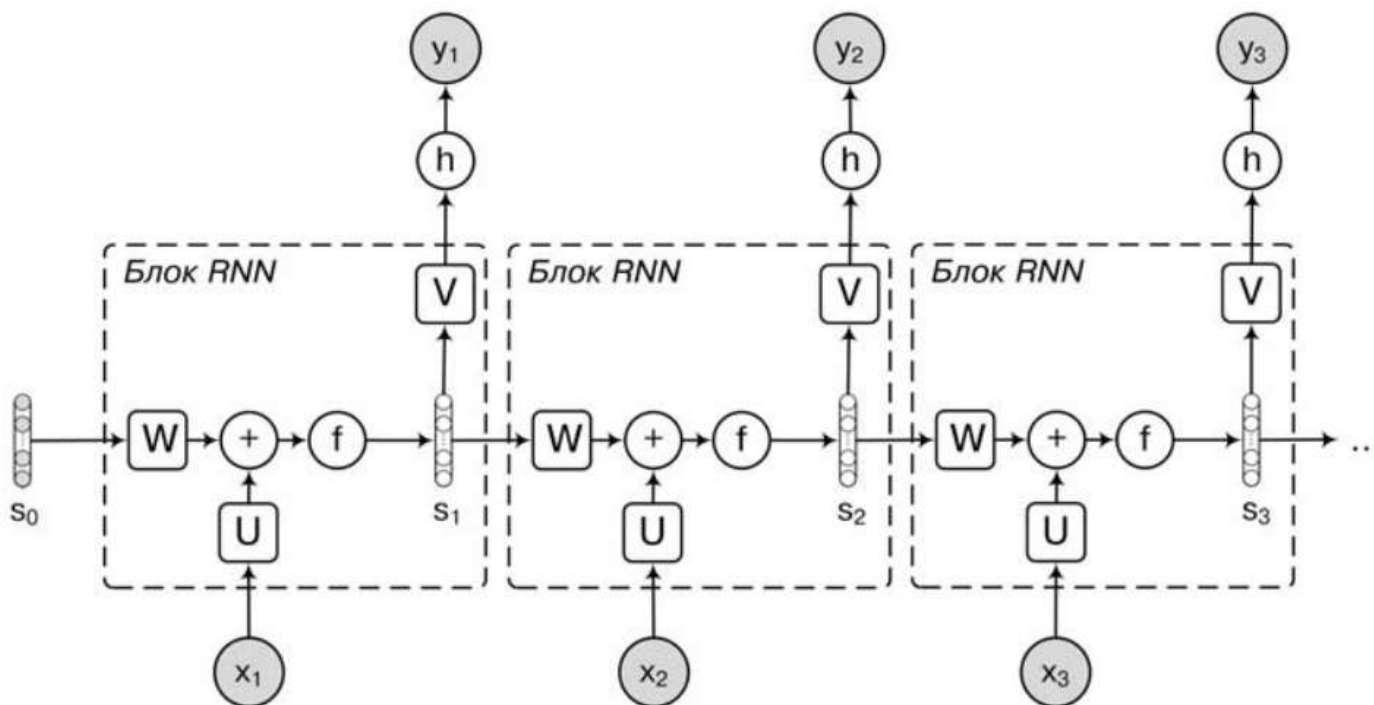
$$\text{softmax}(x)_j = \frac{\exp(x_j)}{\sum_i \exp(x_i)}.$$

5.2 Рекуррентные нейронные сети: простые РНС

- Обозначим через W матрицу весов для перехода между скрытыми состояниями, через U матрицу весов для входов, а через V — для выходов. Архитектура простой рекуррентной нейронной сети (простой РНС, Simple RNN) показана на рисунке ниже. Входы и скрытые состояния будем считать векторами. Будем считать, что на вход всегда подается вектор x ; это не приводит ни к какой потере общности: например, в формуле выше можно было просто переобозначить $x_i = (x_1, x_2, x_3)$. В итоге мы получаем такое формальное задание сети — в момент времени t :

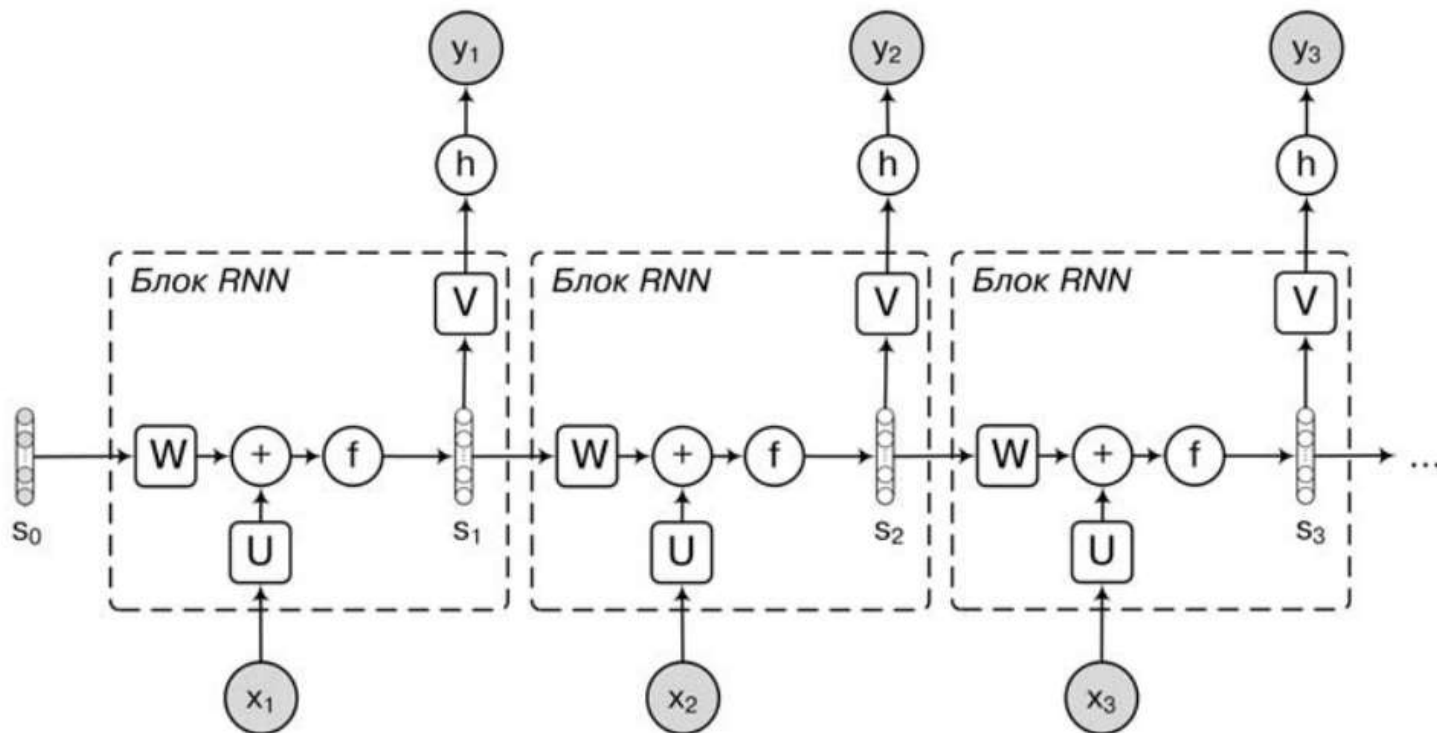
$$\begin{aligned} a_t &= b + Ws_{t-1} + Ux_t, & s_t &= f(a_t), \\ o_t &= c + Vs_t, & y_t &= h(o_t), \end{aligned}$$

- где f — это нелинейность собственно рекуррентной сети (обычно σ , $\tanh$ или ReLU), а h — функция, с помощью которой получается ответ (например, softmax).



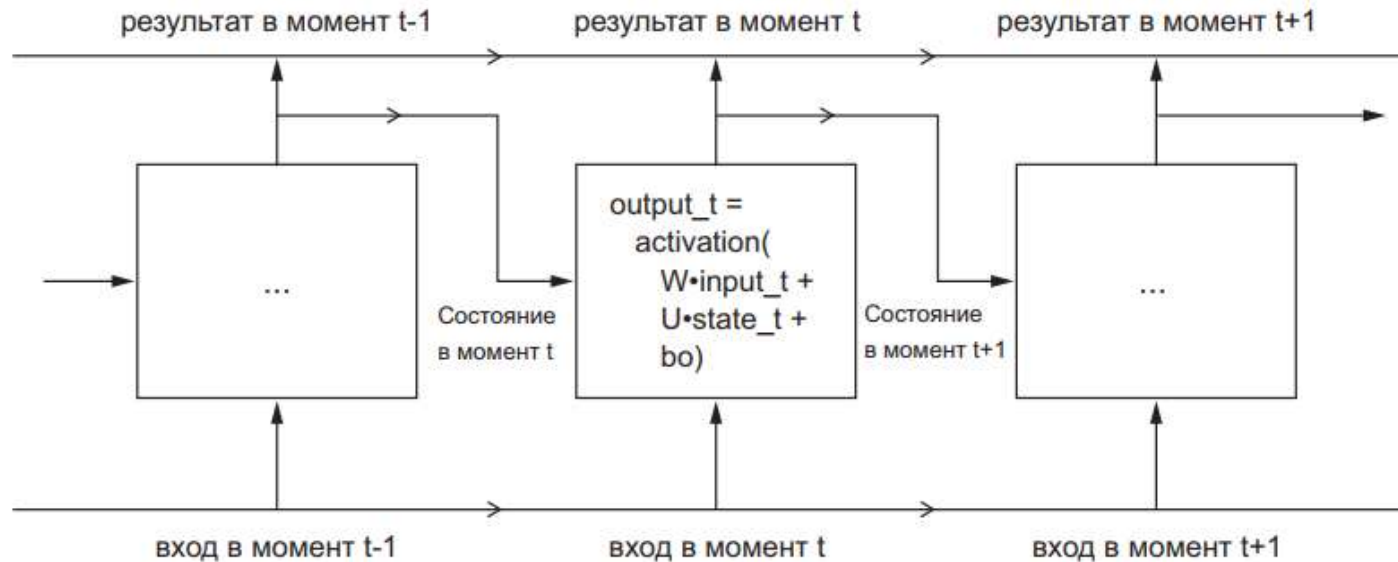
5.2 Рекуррентные нейронные сети: простые РНС

- Будем считать, что прошло T шагов, $t = 1 \dots T$, и у нас задана какая-то функция ошибки $L(\mathbf{o}_1, \dots, \mathbf{o}_T)$. Теперь можно напрямую попытаться подсчитать частные производные, постепенно «разворачивая» внутренние состояния $\mathbf{s}_t$ от $t=T$ обратно к $t = 1$. Получится, что один и тот же вес из матрицы W участвует в этом процессе $T-1$ раз, по числу переходов, но полностью развернутый граф все равно будет, конечно, ациклическим. Получается, что рекуррентная сеть — это как будто очень многоуровневая обычная сеть, в которой одни и те же веса переиспользуются на каждом уровне.



5.2 Рекуррентные нейронные сети: простые РНС.

Реализация «простой» РНС на Python на основе Numpy



□ Реализация RNN в псевдокоде

```
state_t = 0      ← Состояние в момент t
for input_t in input_sequence:  ← Цикл по последовательности элементов
    output_t = activation(dot(W, input_t) + dot(U, state_t) + b)
    state_t = output_t  ← Предыдущее выходное значение становится
                        текущим состоянием для следующей итерации
```

5.2 Рекуррентные нейронные сети: простые РНС.

Реализация «простой» РНС на Python на основе Numpy

Начальное состояние: вектор
с нулевыми значениями элементов

Входные данные: случайный
шум для простоты примера

```
import numpy as np
```

```
timesteps = 100
```

← Число временных интервалов во входной последовательности

```
input_features = 32
```

← Размерность пространства входных признаков

```
output_features = 64
```

← Размерность пространства выходных признаков

```
inputs = np.random.random((timesteps, input_features))
```

→

```
state_t = np.zeros((output_features,))
```

```
W = np.random.random((output_features, input_features))
```

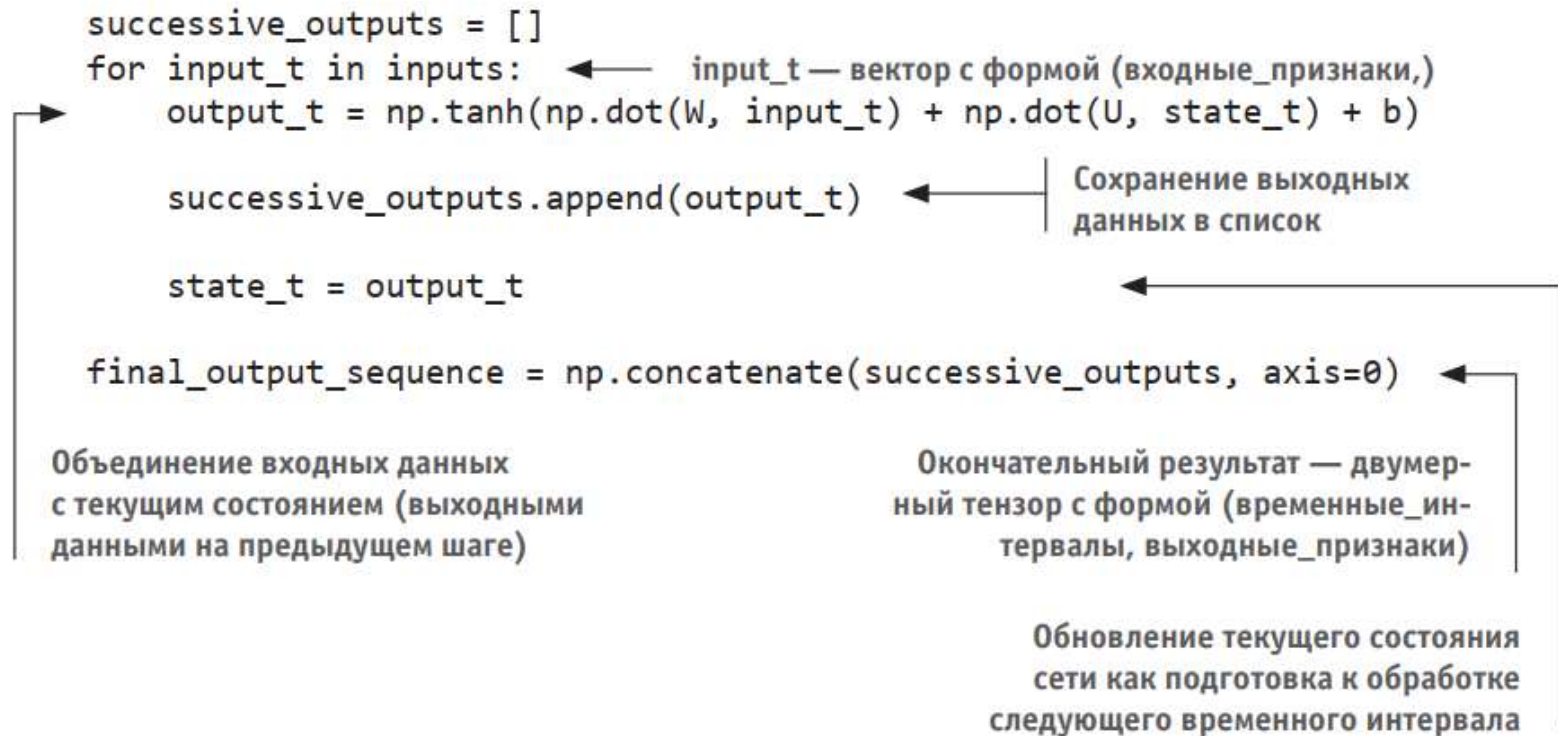
```
U = np.random.random((output_features, output_features))
```

```
b = np.random.random((output_features,))
```

Создание матриц
со случайными
весами

5.2 Рекуррентные нейронные сети: простые РНС.

Реализация «простой» РНС на Python на основе Numpy



Как видите, РНС — это цикл `for`, который повторно использует величины, вычисленные в предыдущей итерации, и не более того. В этом примере конечный результат имеет вид двумерного тензора с формой (временные_интервалы, выходные_признаки), где каждый временной интервал — это результат цикла в момент времени t . Каждому временному интервалу t в выходном тензоре соответствует информация о временных интервалах от 0 до t во входной последовательности — обо всем прошлом.

5.2 Рекуррентные нейронные сети: простые РНС.

Рекуррентный слой SimpleRNN в Keras

- ❑ Процессу, который мы только что реализовали с применением NumPy, соответствует фактический слой в Keras — слой SimpleRNN:

```
from keras.layers import SimpleRNN
```

С одним незначительным отличием: **SimpleRNN** обрабатывает пакеты последовательностей, как и все другие слои в Keras, а не единственную последовательность, как наш предыдущий пример. Это означает, что он принимает входные данные с формой

(размер_пакета/**batch_size**, временные_интервалы/**timesteps**, входные_признаки/**input_features**), а не (временные_интервалы/**timesteps**, входные_признаки/**input_features**).

- ❑ Подобно всем рекуррентным слоям в Keras, SimpleRNN может действовать в двух разных режимах: возвращать полные последовательности результатов для всех временных интервалов — трехмерный тензор с формой

(размер_пакета/**batch_size**, временные_интервалы/**timesteps**, выходные_признаки/**output_features|units**)

или только последний результат для каждой входной последовательности — двумерный тензор с формой

(размер_пакета/**batch_size**, выходные_признаки/**output_features|units**).

Здесь **output_features|units** – размерность выходного пространства.

Выбор режима управляется аргументом **return_sequences** конструктора: **True** – вся последовательность, **False** (значение по умолчанию) – последний экземпляр. Это работает для всех типов рекуррентных слоев в Keras.

5.2 Рекуррентные нейронные сети: простые РНС.

Рекуррентный слой SimpleRNN в Keras

- ❑ Рассмотрим пример, в котором используется слой SimpleRNN и возвращается результат только для последнего временного интервала:

```
num_features = 14
steps = 120
inputs = keras.Input(shape=(steps, num_features))
outputs = layers.SimpleRNN(16)(inputs)
print(outputs.shape)
model = Model(inputs, outputs)
model.summary()
```

(None, 16)

Model: "model"

Layer (type)	Output Shape	Param #
=====		
input_8 (InputLayer)	[(None, 120, 14)]	0
simple_rnn_11 (SimpleRNN)	(None, 16)	496

=====

Total params: 496 (1.94 KB)
Trainable params: 496 (1.94 KB)
Non-trainable params: 0 (0.00 Byte)

5.2 Рекуррентные нейронные сети: простые РНС.

Рекуррентный слой SimpleRNN в Keras

❑ Следующий пример возвращает полную последовательность состояний:

```
num_features = 14
steps = 120
inputs = keras.Input(shape=(steps, num_features))
outputs = layers.SimpleRNN(16, return_sequences=True)(inputs)
print(outputs.shape)
model = Model(inputs, outputs)
model.summary()
```

```
(None, 120, 16)
Model: "model_1"
```

Layer (type)	Output Shape	Param #
=====		
input_9 (InputLayer)	[(None, 120, 14)]	0
simple_rnn_12 (SimpleRNN)	(None, 120, 16)	496
=====		
Total params: 496 (1.94 KB)		
Trainable params: 496 (1.94 KB)		
Non-trainable params: 0 (0.00 Byte)		

5.2 Рекуррентные нейронные сети: простые РНС.

Рекуррентный слой SimpleRNN в Keras

- ❑ Иногда полезно наложить друг на друга несколько рекуррентных слоев, чтобы увеличить репрезентативность сети. В таких ситуациях все промежуточные слои должны возвращать полные последовательности результатов:

```
num_features = 14
steps = 120
inputs = keras.Input(shape=(steps, num_features))
x = layers.SimpleRNN(16, return_sequences=True)(inputs)
x = layers.SimpleRNN(16, return_sequences=True)(x)
outputs = layers.SimpleRNN(16)(x) # Последний слой возвращает только последний результат
print(outputs.shape)
model = Model(inputs, outputs)
model.summary()
```

(None, 16)

Model: "model_2"

Layer (type)	Output Shape	Param #
=====		
input_11 (InputLayer)	[(None, 120, 14)]	0
simple_rnn_16 (SimpleRNN)	(None, 120, 16)	496
simple_rnn_17 (SimpleRNN)	(None, 120, 16)	528
simple_rnn_18 (SimpleRNN)	(None, 16)	528

=====

Total params: 1552 (6.06 KB)
Trainable params: 1552 (6.06 KB)
Non-trainable params: 0 (0.00 Byte)

5.2 Рекуррентные нейронные сети: простые РНС.

Рекуррентный слой SimpleRNN в Keras

- ❑ Иногда полезно наложить друг на друга несколько рекуррентных слоев, чтобы увеличить репрезентативность сети. В таких ситуациях все промежуточные слои должны возвращать полные последовательности результатов:

```
num_features = 14
steps = 120
inputs = keras.Input(shape=(steps, num_features))
x = layers.SimpleRNN(16, return_sequences=True)(inputs)
x = layers.SimpleRNN(16, return_sequences=True)(x)
# Последний слой возвращает всю последовательность
outputs = layers.SimpleRNN(16, return_sequences=True)(x)
print(outputs.shape)
model = Model(inputs, outputs)
model.summary()
```

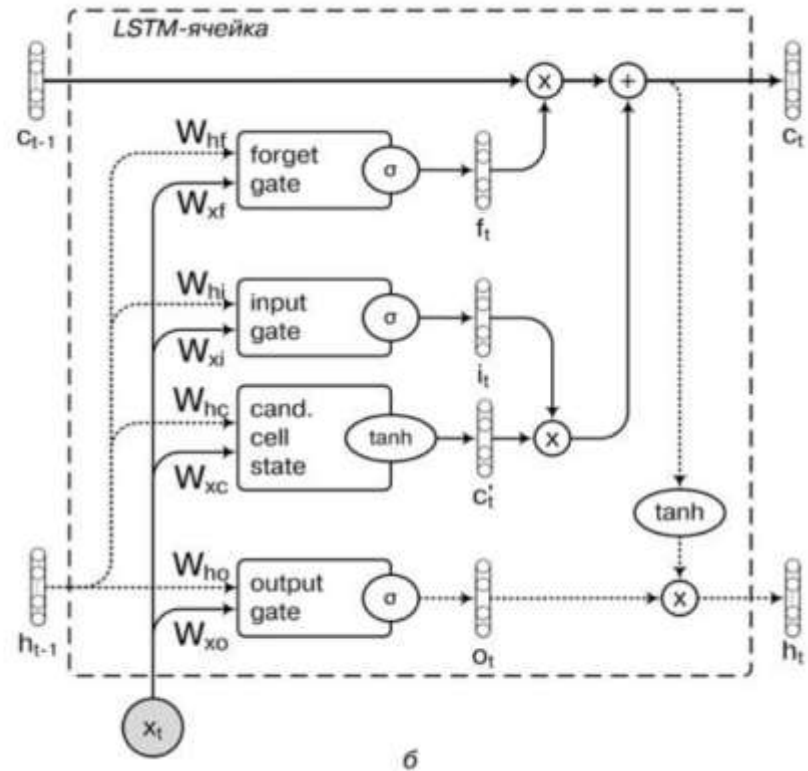
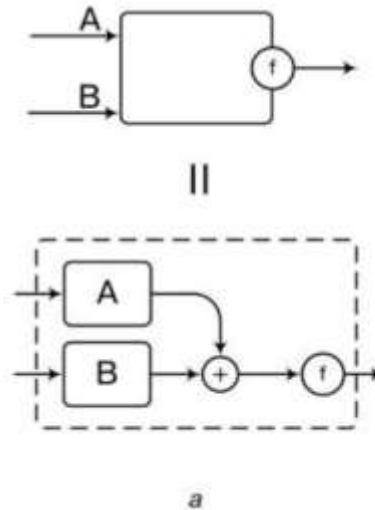
```
(None, 120, 16)
Model: "model_3"
```

Layer (type)	Output Shape	Param #
=====		
input_13 (InputLayer)	[(None, 120, 14)]	0
simple_rnn_22 (SimpleRNN)	(None, 120, 16)	496
simple_rnn_23 (SimpleRNN)	(None, 120, 16)	528
simple_rnn_24 (SimpleRNN)	(None, 120, 16)	528

```
=====
Total params: 1552 (6.06 KB)
Trainable params: 1552 (6.06 KB)
Non-trainable params: 0 (0.00 Byte)
```

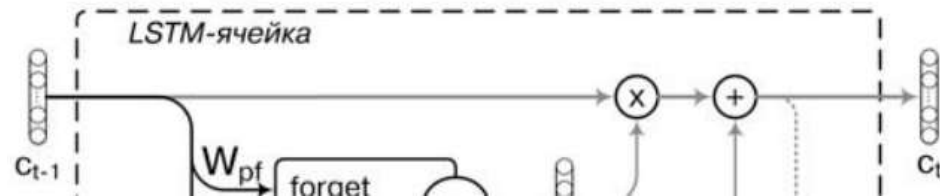
5.3 РНС типа LSTM - Long Short-Term Memory

- ❑ Простые РНС очень плохо справляются с ситуациями, когда нужно что-то «запомнить» надолго: влияние скрытого состояния или входа с шага t на последующие состояния рекуррентной сети экспоненциально затухает. Проблема была глубоко изучена Хохрейтером (Hochreiter) (1991), Бенджио и др. (1994).
- ❑ Решения, которые на данный момент предлагаются, состоят главным образом в том, чтобы изменить, усложнить архитектуру одной ячейки – «кирпичика» рекуррентной сети. Одна из самых широко известных и часто применяющихся конструкций таких ячеек — это LSTM (от слов *Long Short-Term Memory* – «долгая краткосрочная память»).
- ❑ Предложены Хохрейтером и Шмидхубером (Schmidhuber) (1997).
- ❑ LSTM: *а* — обозначение гейта с двумя входами; *б* — структура LSTM-ячейки.
- ❑ В LSTM есть три основных вида узлов, которые называются гейтами: входной (input gate), забывающий (forget gate) и выходной (output gate), а также собственно рекуррентная ячейка со скрытым состоянием.

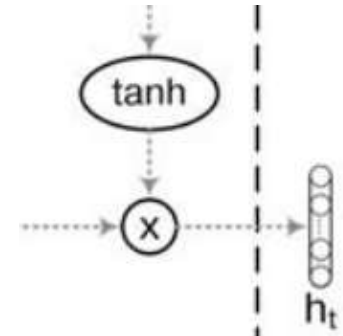
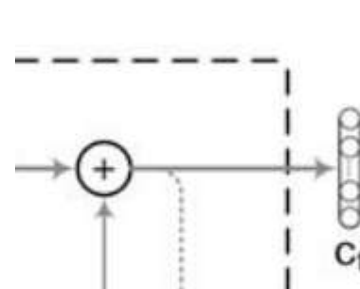
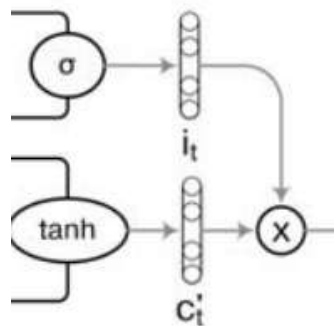
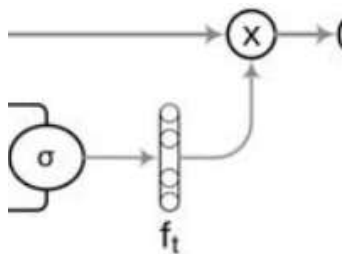


5.3 РНС типа LSTM - Long Short-Term Memory

- Ключ к пониманию функционирования LSTM - клеточное состояние (cell state) - горизонтальная линия, проходящая сквозь верхнюю часть схемы. Клеточное состояние - это что-то типа ленты конвейера. Она движется прямо вдоль всей цепи только лишь с небольшими линейными взаимодействиями. Информация может просто течь по ней без изменений или же меняться в соответствии с входным потоком.



- LSTM имеет способность удалять или добавлять информацию к состоянию ячейки, однако эта способность тщательно регулируется структурами, называемыми вентилями (**gates**). Вентили - это способ избирательно пропускать информацию. Они составлены из сигмоидного или тангенциального слоя и операции поточечного умножения (pointwise multiplication). Результаты в свою очередь далее объединяются через поточечные операции для формирования нового состояния ячейки сети и выхода ячейки сети.



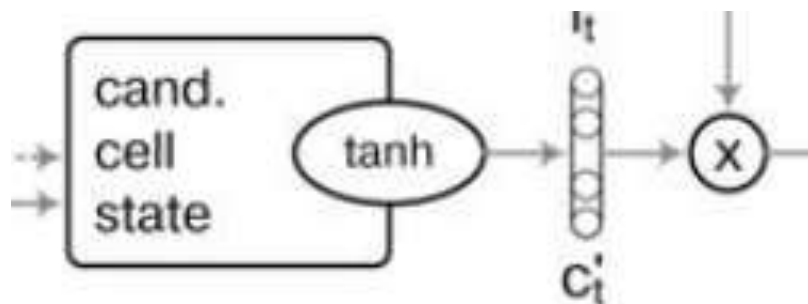
5.3 РНС типа LSTM - Long Short-Term Memory

□ Если обозначить через $\mathbf{x}_t$ входной вектор во время t , через $\mathbf{h}_t$ — вектор скрытого состояния во время t , через W_i (с разными вторыми индексами) — матрицы весов, применяющиеся ко входу, через W_h — матрицы весов в рекуррентных соединениях, а через $\mathbf{b}$ — векторы свободных членов, мы получим следующее формальное определение того, как работает LSTM: на очередном входе $\mathbf{x}_t$, имея скрытое состояние из предыдущего шага $\mathbf{h}_{t-1}$ и собственно состояние ячейки $\mathbf{c}_{t-1}$, мы последовательно вычисляем

$$\begin{aligned} \mathbf{c}'_t &= \tanh(W_{xc}\mathbf{x}_t + W_{hc}\mathbf{h}_{t-1} + \mathbf{b}_{c'}) && \text{candidate cell state} \\ \mathbf{i}_t &= \sigma(W_{xi}\mathbf{x}_t + W_{hi}\mathbf{h}_{t-1} + \mathbf{b}_i) && \text{input gate} \\ \mathbf{f}_t &= \sigma(W_{xf}\mathbf{x}_t + W_{hf}\mathbf{h}_{t-1} + \mathbf{b}_f) && \text{forget gate} \\ \mathbf{o}_t &= \sigma(W_{xo}\mathbf{x}_t + W_{ho}\mathbf{h}_{t-1} + \mathbf{b}_o) && \text{output gate} \\ \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{c}'_t, && \text{cell state} \\ \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t) && \text{block output} \end{aligned}$$

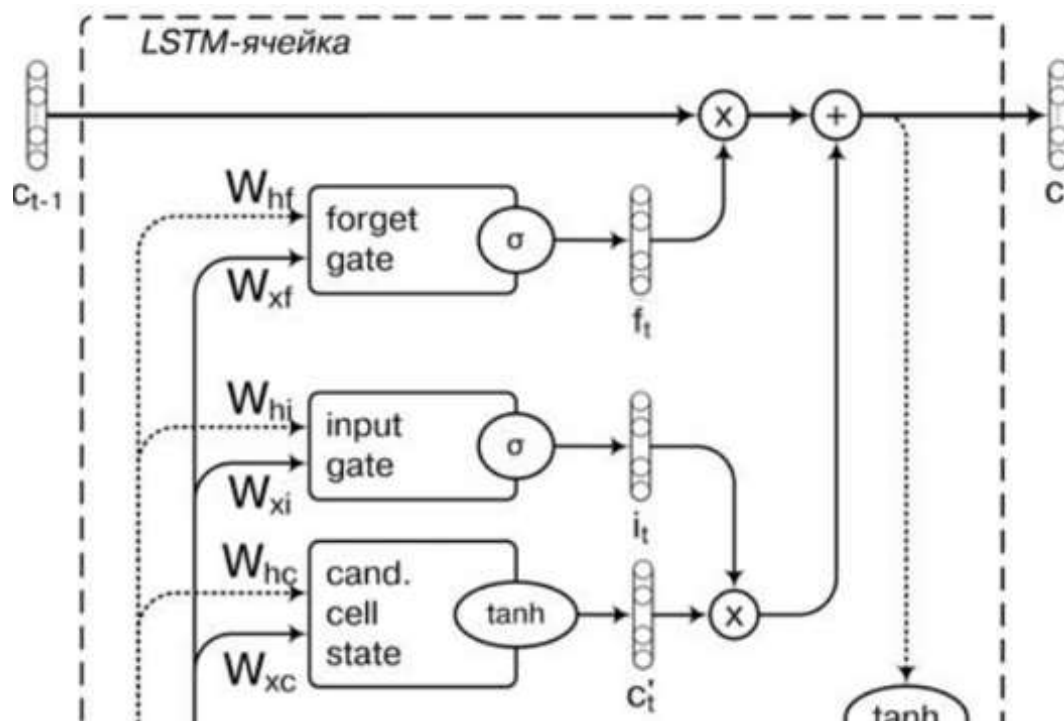
5.3 РНС типа LSTM - Long Short-Term Memory

- На вход LSTM, как и в «обычной» РНС, подаются два вектора: новый вектор из входных данных x_t и вектор скрытого состояния h_{t-1} , который получен из скрытого состояния этой ячейки на предыдущем шаге. Кроме того, внутри у каждого LSTM-блока есть «ячейка памяти» (cell) — вектор, который выполняет функцию памяти. Вектор ячейки на шаге t мы обозначили выше через c_t , а c'_t , который получается в первом же уравнении, — это вектор, полученный из входа и предыдущего скрытого состояния, который становится кандидатом на новое значение памяти. Получается он из x_t и h_{t-1} весьма обычным для нейронных сетей преобразованием: сначала линейная функция, потом гиперболический тангенс, все как в обычных нейронных сетях.



5.3 РНС типа LSTM - Long Short-Term Memory

- ❑ Но c'_t — это всего лишь кандидат в новое значение памяти. Прежде чем его запишут на место c_{t-1} , значение-кандидат и старое значение проходят через еще два гейта: входной гейт i_t и забывающий гейт f_t .
- ❑ Новое значение c_t получается как линейная комбинация из старого с коэффициентами из забывающего гейта f_t и нового кандидата c'_t с коэффициентами из входного гейта i_t . Там, где значения вектора забывающего гейта f_t будут близки к нулю, старое значение c_{t-1} «забудется», и где значения i_t будут велики, новый входной вектор прибавится к тому, что было в памяти. И наоборот – где значения вектора забывающего гейта f_t будут близки к 1, старое значение c_{t-1} «сохранится».

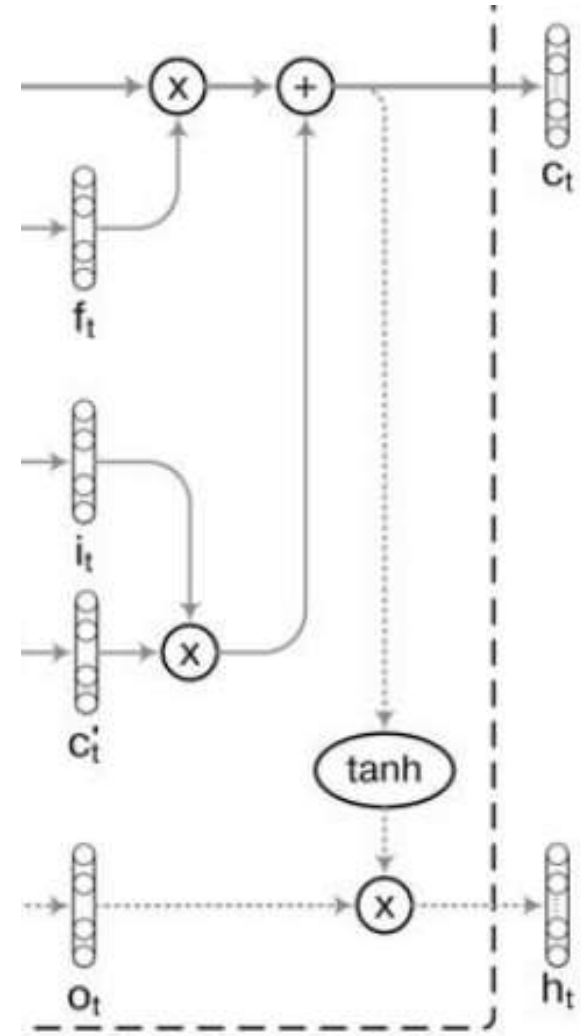


5.3 РНС типа LSTM - Long Short-Term Memory

- ❑ Сигмоидный слой подает на выход числа между нулем и единицей, описывая таким образом, насколько каждый компонент должен быть пропущен сквозь вентиль. Ноль - “ничего не пропускать”, один - “пропускать все”. LSTM имеет три таких вентиля, чтобы защищать и контролировать состояние памяти ячейки.
- ❑ Обратите внимание: покомпонентное умножение приводит к тому, что на очередном шаге может быть перезаписана только часть «памяти» LSTM-ячейки; и какая это будет часть, тоже определяет сама ячейка в зависимости от того, что получается на выходах забывающего гейта f_t и входного гейта i_t . Более того, поскольку эта линейная комбинация «мягкая» и все пропускается через сигмoиды σ , LSTM-ячейка может не просто выбрать, записать новое значение или выкинуть его, а еще и сохранить любую линейную комбинацию старого и нового значения, причем коэффициенты могут быть разными в разных компонентах вектора; и эти решения ячейка принимает в зависимости от конкретного входа. Все это делает LSTM-ячейки весьма гибкими.

5.3 РНС типа LSTM - Long Short-Term Memory

- ❑ Наконец, нам нужно решить, какой результат мы собираемся подать на выход h_t . Этот результат будет основан на полученном состоянии памяти ячейки c_t , но будет его отфильтрованной сквозь гиперболический тангенс (**tanh**) версией (чтобы уместить значения в промежуток от -1 до 1). Умножаем полученное значение на выход сигмоидного вентиля o_t , так что мы отправляем на выход только те части, которые мы хотим.
- ❑ LSTM благодаря своей архитектуре решают проблему исчезающих градиентов, которая мешала рекуррентным сетям анализировать долгосрочные зависимости.

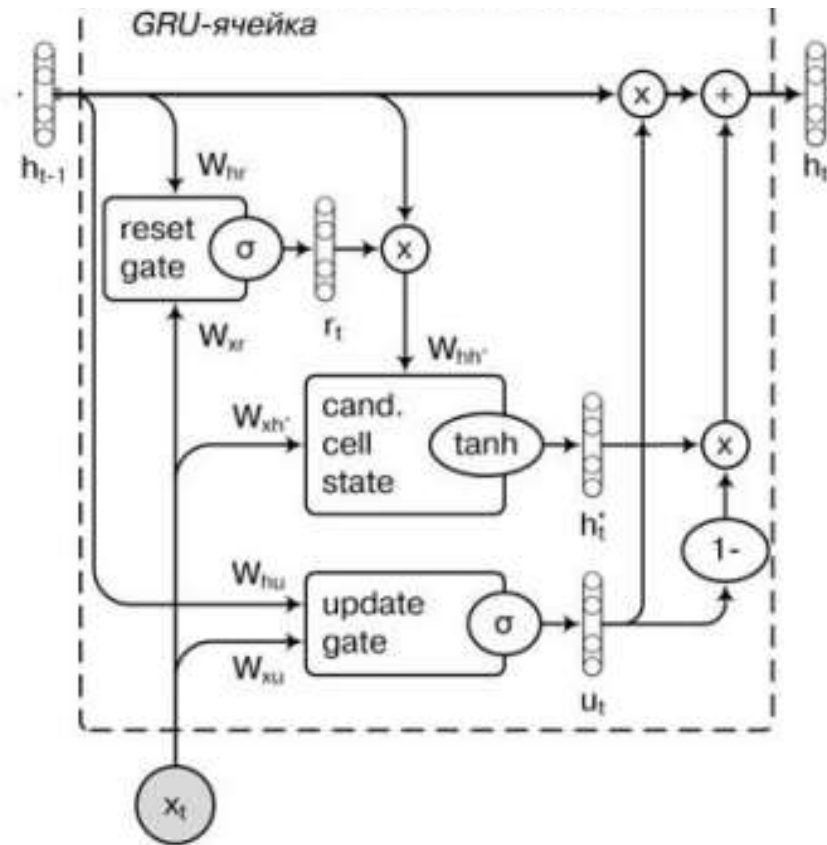


5.4 РНС типа Gated Recurrent Units или GRU RNN

- В работах 2014 года группой известных специалистов в области глубокого обучения Kyunghyun Cho и Yoshua Bengio (и др.) была предложена модификация LSTM рекуррентных сетей – Gated Recurrent Units или GRU RNN. В этой архитектуре используется как раз идея совмещения выходного и забывающего гейта, а скрытое состояние h_t совмещено со значением памяти c_t . Вот как работает одна GRU-ячейка:

$$\begin{aligned}u_t &= \sigma(W_{xu}x_t + W_{hu}h_{t-1} + b_u), \\r_t &= \sigma(W_{xr}x_t + W_{hr}h_{t-1} + b_r), \\h'_t &= \tanh(W_{xh'}x_t + W_{hh'}(r_t \odot h_{t-1})), \\h_t &= (1 - u_t) \odot h'_t + u_t \odot h_{t-1}.\end{aligned}$$

- Здесь u_t — это гейт обновления (update gate), который и является комбинацией входного и забывающего гейтов. А r_t — это гейт перезагрузки (reset gate); он тоже отвечает за то, какую часть памяти нужно перенести дальше с прошлого шага, но делает это еще до применения нелинейной функции. Ячейка памяти и выход блока h_t тут, в отличие от LSTM, никак не разделяются, и следующий выход h_t получается как комбинация (задаваемая гейтом u_t) предыдущего выхода h_{t-1} и текущего кандидата в выход h'_t , который, в свою очередь, тоже зависит от h_{t-1} , но на этот раз через гейт перезагрузки r_t .



5.4 PHC типа Gated Recurrent Units или GRU RNN

- ❑ Практика показывает, что GRU практически всегда работает так же или почти так же хорошо, как LSTM. А параметров получается гораздо меньше: шесть матриц весов против восьми. Из-за меньшего числа параметров тренировать GRU проще, чем LSTM, поэтому в некоторых задачах GRU показывает результаты лучше. Более того, можно позволить себе уместить в той же памяти и с теми же ресурсами сети из большего числа GRU-ячеек, чем LSTM. Поэтому GRU часто используются вместо классических LSTM.

5.6 Примеры программных реализаций RNN.

Классификация данных телеметрии МКА

Экспериментальные данные ТМИ МКА

- ❑ Для проведения компьютерных экспериментов и анализа точности разработанных нейросетевых классификационных моделей использованы данные ТМИ навигационной подсистемы МКА.
- ❑ Вектор матрицы ТМИ X_i имеет размерность 9 и помечается 0 в случае штатного состояния и 1 в случае нештатного состояния подсистемы. Общая размерность 9-мерного временного ряда X составляет 121690 векторов, из которых 77881 векторов составляет обучающий набор данных, 19471 векторов составляет валидационный набор данных, 24338 вектор составляет тестовый набор данных.

5.6 Примеры программных реализаций RNN.

Классификация данных телеметрии МКА

❑ Подготовка данных:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import tensorflow as tf
import keras
```

Using TensorFlow backend.

```
data = pd.read_csv("d:\\PythonProjects\\Reliability_analysis\\June_2017_1_121691.csv")
data_arr = data.values
size = data_arr.shape
x = data_arr[:,0:size[1]-1]
y = data_arr[:,size[1]-1]
x = x.reshape((x.shape[0], x.shape[1], 1))
y = y.reshape((y.shape[0], 1))

from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=.2)
x_train, x_val, y_train, y_val = train_test_split(x_train, y_train, test_size=.25)
```

5.6 Примеры программных реализаций RNN.

Классификация данных телеметрии МКА

- ❑ Создание и компиляция модели нейронной сети на основе слоев LSTM:

```
import time

model = keras.models.Sequential()
model.add(keras.layers.LSTM(units=64, return_sequences=True, input_shape=(x.shape[1], 1)))
model.add(keras.layers.LSTM(units=32, return_sequences=True, kernel_regularizer='l2'))
model.add(keras.layers.LSTM(units=16, kernel_regularizer='l2'))
model.add(keras.layers.Dense(1, activation='sigmoid'))

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

start_time = time.time()

model.fit(x_train, y_train, validation_data=(x_val, y_val), epochs=500, batch_size=128)

duration = time.time() - start_time
print('Duration time:', duration)

test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test_loss: {a:.4f} Test_acc: {b:.4f}".format(a=test_loss, b=test_acc))
```

5.6 Примеры программных реализаций RNN.

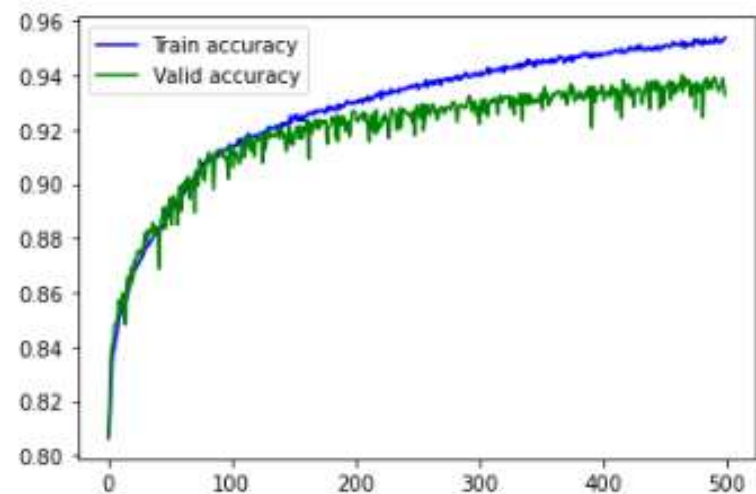
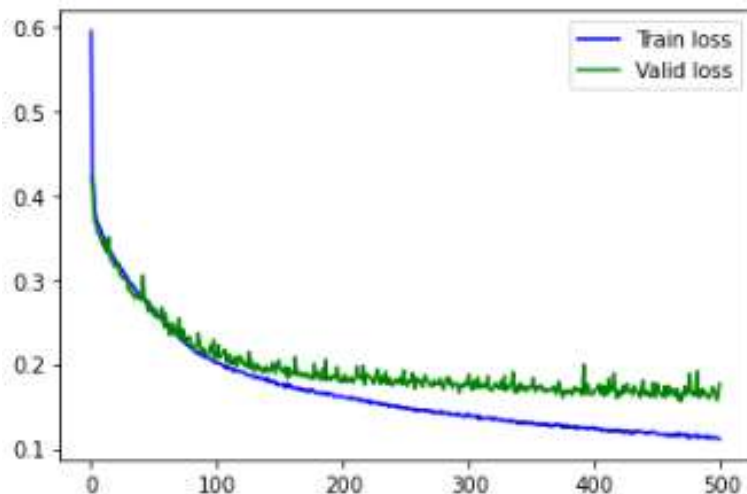
Классификация данных телеметрии МКА

❑ Вывод графиков и результатов качества модели LSTM нейронной сети:

```
plt.plot(model.history.history['loss'], color="b", label='Train loss')
plt.plot(model.history.history['val_loss'], color="g", label='Valid loss')
plt.legend()
plt.show()

plt.plot(model.history.history['acc'], color="b", label='Train accuracy')
plt.plot(model.history.history['val_acc'], color="g", label='Valid accuracy')
plt.legend()
plt.show()
```

Epoch 500/500
73014/73014 [=====] - 9s 125us/step - loss: 0.1113 - acc: 0.9538 - val_loss: 0.1770 - val_acc: 0.9327
Duration time: 4615.746706008911
24338/24338 [=====] - 2s 74us/step
Test_loss: 0.1770 Test_acc: 0.9327



5.6 Примеры программных реализаций RNN.

Классификация данных телеметрии МКА

- ❑ Создание и компиляция модели нейронной сети на основе слоев GRU:

```
import time

model = keras.models.Sequential()
model.add(keras.layers.GRU(units=64, return_sequences=True, input_shape=(x.shape[1], 1)))
model.add(keras.layers.GRU(units=32, return_sequences=True, kernel_regularizer='l2'))
model.add(keras.layers.GRU(units=16, kernel_regularizer='l2'))
model.add(keras.layers.Dense(1, activation='sigmoid'))

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

start_time = time.time()

model.fit(x_train, y_train, validation_data=(x_val, y_val), epochs=500, batch_size=128)

duration = time.time() - start_time
print('Duration time:', duration)

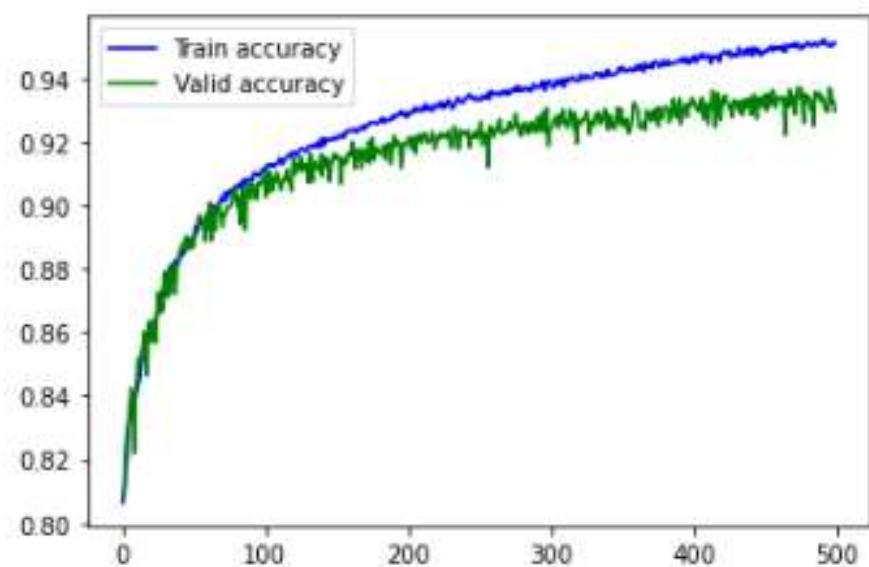
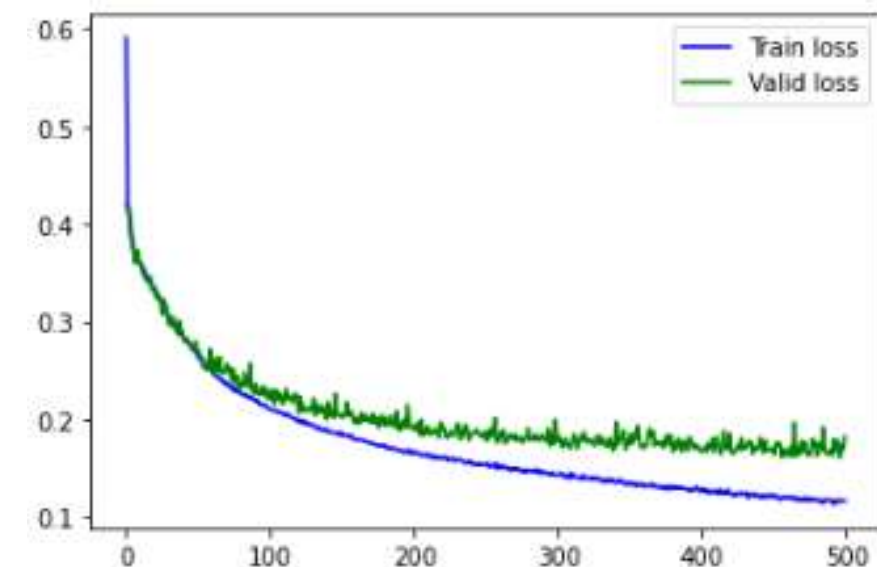
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test_loss: {a:.4f} Test_acc: {b:.4f}".format(a=test_loss, b=test_acc))
```

5.6 Примеры программных реализаций RNN.

Классификация данных телеметрии МКА

❑ Вывод графиков и результатов качества модели нейронной сети:

```
Epoch 500/500
73014/73014 [=====] - 7s 102us/step - loss: 0.1145 - acc: 0.9527 - val_loss: 0.1619 - val_acc: 0.9372
Duration time: 3656.853282213211
24338/24338 [=====] - 1s 58us/step
Test_loss: 0.1581 Test_acc: 0.9381
```



5.6 Примеры программных реализаций RNN.

Классификация данных телеметрии МКА. Сравнение с CNN

□ Давайте рассмотрим опять применение 1-мерной сверточной сети для данной задачи :

```
import time

from keras.models import Sequential
from keras import layers
from keras.optimizers import Adam
model = Sequential()
model.add(layers.Conv1D(filters=256, kernel_size=4, padding='same', activation='relu',
input_shape=(x.shape[1], 1)))
model.add(layers.MaxPooling1D(2))
model.add(layers.Conv1D(filters=128, kernel_size=2, padding='same', activation='relu'))
model.add(layers.GlobalMaxPooling1D())
model.add(layers.Dense(1, activation='sigmoid'))

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

start_time = time.time()

model.fit(x_train, y_train, validation_data=(x_val, y_val), epochs=500, batch_size=128)

duration = time.time() - start_time
print('Duration time:', duration)

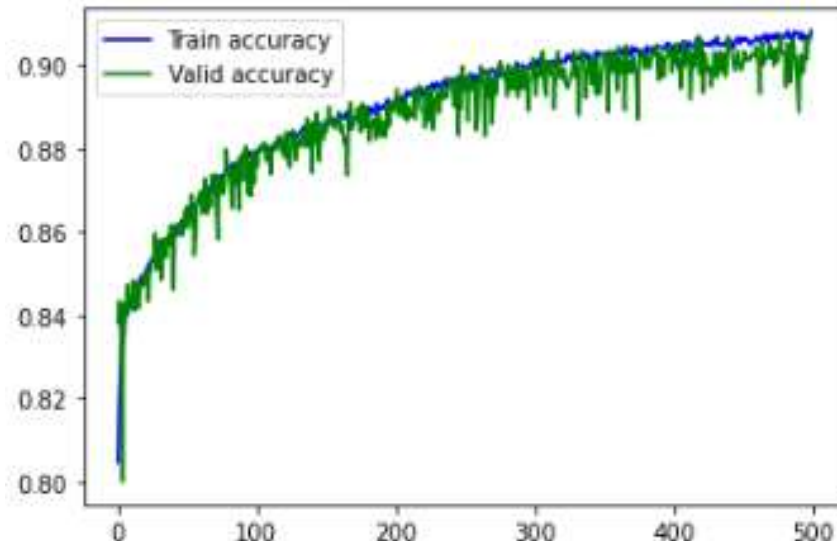
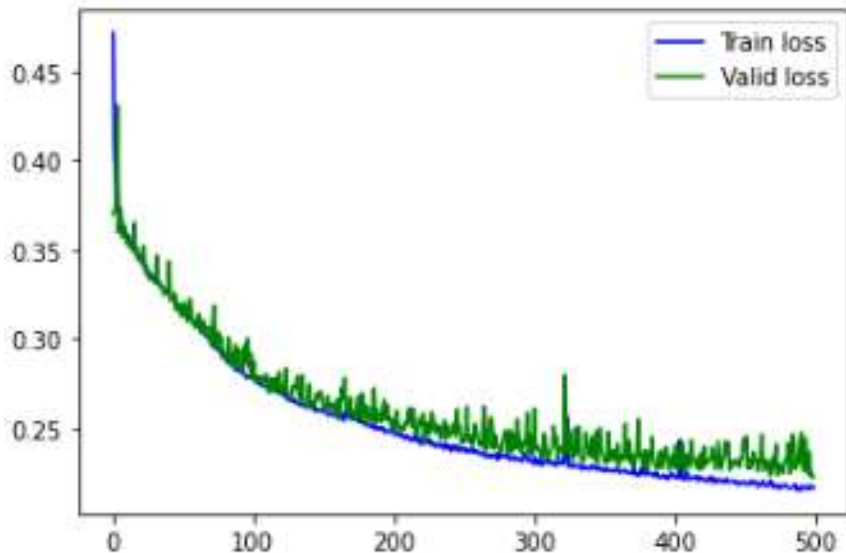
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test_loss: {a:.4f} Test_acc: {b:.4f}".format(a=test_loss, b=test_acc))
```

5.6 Примеры программных реализаций RNN.

Классификация данных телеметрии МКА. Сравнение с CNN

❑ Вывод графиков и результатов качества модели нейронной сети:

```
Epoch 500/500  
73014/73014 [=====] - 3s 46us/step - loss: 0.2161 - acc: 0.9083 - val_loss: 0.2222 - val_acc: 0.9075  
Duration time: 1688.0928621292114  
24338/24338 [=====] - 1s 22us/step  
Test_loss: 0.2131 Test_acc: 0.9149
```



5.6 Примеры программных реализаций RNN.

Классификация данных телеметрии МКА. Сравнение с CNN

- Результаты решения данной задачи, как говорилось ранее, показали, что РНС на основе слоев GRU работает в данном случае так же хорошо, как РНС LSTM. А параметров получается гораздо меньше: шесть матриц весов против восьми. Из-за меньшего числа параметров тренировать РНС GRU понадобилось время на обучение и валидацию существенно меньше, чем РНС LSTM: 3657 сек. < 4616 сек. При этом уровень точности на этапе валидации и тестирования совпадает. Поэтому в данной задаче применение РНС GRU предпочтительнее. Поэтому GRU часто используются вместо классических LSTM.
- Если же сравнить результаты, полученные 1-мерной сверточной сетью на тех же 500-х эпохах обучения и валидации, то с одной стороны точность на этапе тестирования $0.92 < 0.94$ в случае РНС GRU или LSTM. С другой стороны – время обучения и валидации существенно меньше – 1688 сек., и наблюдается гораздо меньший эффект переобучения. Поэтому в случае, когда такой уровень точности классификации удовлетворителен на практике, 1-мерная сверточная сеть может быть хорошей альтернативой РНС для классификационного анализа данных ТМИ и других последовательностей.

5.6 Примеры программных реализаций RNN.

Регрессионный анализ

- Пример решения задачи регрессионного анализа будем рассматривать на искусственно сгенерированном наборе данных, состоящем из 200 векторов размерностью 20 признаков, с одним целевым признаком. Набор был разбит на обучающую валидационную и тестовую выборки в соотношении: 0.6, 0.2, 0.2.

```
from sklearn.datasets import make_regression
```

```
x, y = make_regression(n_samples=200,  
                      n_features=20,  
                      n_informative=20,  
                      n_targets=1,  
                      random_state=1972)
```

x.shape

(200, 20)

y.shape

(200,)

```
import numpy as np  
import matplotlib.pyplot as plt  
from sklearn.model_selection import train_test_split  
test_size = 0.2  
val_size = 0.25  
seed = 1972  
y = np.reshape(y, (-1,1))  
X_train, X_test, Y_train, Y_test = train_test_split(x, y,\br/>test_size=test_size, random_state=seed)  
X_train, X_val, Y_train, Y_val = train_test_split(X_train, Y_train,\br/>test_size=val_size, random_state=seed)
```

5.6 Примеры программных реализаций RNN.

Регрессионный анализ. Модель полносвязной сети.

- Для сравнения сначала рассмотрим регрессионную модель полносвязной сети, которая ранее применялась для одномерного набора данных (X,Y).

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import models
from tensorflow.keras import layers
from tensorflow.keras import optimizers

from keras import Input
active1 = 'elu' #'leaky_relu'
active2 = 'linear'
model = keras.models.Sequential()
model.add(layers.Dense(128, activation=active1, input_shape=(x.shape[1],)))
model.add(layers.Dense(256, activation=active1))
model.add(layers.Dense(512, activation=active1))
model.add(layers.Dense(256, activation=active1))
model.add(layers.Dense(128, activation=active1))
model.add(layers.Dense(56, activation=active1))
model.add(keras.layers.Dense(1, activation=active2))

model.summary()
```

5.6 Примеры программных реализаций RNN.

Регрессионный анализ. Модель полносвязной сети.

- Для сравнения сначала рассмотрим регрессионную модель полносвязной сети, которая ранее применялась для одномерного набора данных (X,Y).

```
model.summary()
```

```
Model: "sequential_61"
```

Layer (type)	Output Shape	Param #
dense_245 (Dense)	(None, 128)	2688
dense_246 (Dense)	(None, 256)	33024
dense_247 (Dense)	(None, 512)	131584
dense_248 (Dense)	(None, 256)	131328
dense_249 (Dense)	(None, 128)	32896
dense_250 (Dense)	(None, 56)	7224
dense_251 (Dense)	(None, 1)	57

```
=====  
Total params: 338801 (1.29 MB)
```

```
Trainable params: 338801 (1.29 MB)
```

```
Non-trainable params: 0 (0.00 Byte)
```

5.6 Примеры программных реализаций RNN.

Регрессионный анализ. Модель полносвязной сети.

- ❑ Для сравнения сначала рассмотрим регрессионную модель полносвязной сети, которая ранее применялась для одномерного набора данных (X,Y).
- ❑ Список callbacks:

```
model.compile(optimizer='adam',  
loss='mse',  
metrics= [tf.keras.metrics.RootMeanSquaredError()])  
  
callbacks_list = [  
    keras.callbacks.EarlyStopping(  
        monitor='val_loss',  
        patience=25,  
    ),  
    keras.callbacks.ModelCheckpoint(  
        filepath='Dense_Regression_30.09.2024.h5',  
        monitor='val_loss',  
        save_best_only=True,  
    ),  
    keras.callbacks.ReduceLROnPlateau(  
        monitor='val_loss',  
        factor=0.25,  
        patience=10,  
    )  
]
```

5.6 Примеры программных реализаций RNN.

Регрессионный анализ. Модель полносвязной сети.

❑ Результат обучения с валидацией:

```
m=model.fit(X_train, Y_train, validation_data=(X_val, Y_val), callbacks=callbacks_list,  
            epochs=250, batch_size=16)
```

Epoch 133/250

```
8/8 [=====] - 0s 10ms/step - loss: 0.4361 - root_mean_squared_error: 0.660  
4 - val_loss: 576.2765 - val_root_mean_squared_error: 24.0058 - lr: 6.2500e-05
```

5.6 Примеры программных реализаций RNN.

Регрессионный анализ. Модель полносвязной сети.

❑ Результат тестирования обученной полносвязной модели:

```
import random
from sklearn.metrics import mean_squared_error, r2_score
```

```
test_loss, test_acc = model.evaluate(X_test, Y_test)
print('test_loss:', test_loss, ' ', 'test_acc:', test_acc)
```

```
Y_test_pred = model.predict(X_test)
rmse = np.sqrt(mean_squared_error(Y_test, Y_test_pred))
r2 = r2_score(Y_test, Y_test_pred)
```

```
print(rmse)
print(r2)
```

```
test_loss: 606.7587890625    test_acc: 24.632474899291992
2/2 [=====] - 0s 3ms/step
24.632481726546647
0.9927993500736225
```

5.6 Примеры программных реализаций RNN.

Регрессионный анализ. Модель RNN сети.

- ❑ Рассмотрим регрессионную модель полносвязной сети, которая ранее применялась для одномерного набора данных (X,Y) с добавлением одного рекуррентного слоя.

```
active1 = 'elu'  
active2 = 'linear'  
model = keras.models.Sequential()  
model.add(layers.LSTM(128, return_sequences=True, input_shape=(x.shape[1],1)))  
model.add(layers.Flatten())  
model.add(layers.Dense(256, activation=active1))  
model.add(layers.Dense(512, activation=active1))  
model.add(layers.Dense(256, activation=active1))  
model.add(layers.Dense(128, activation=active1))  
model.add(layers.Dense(56, activation=active1))  
model.add(keras.layers.Dense(1, activation=active2)) |
```

5.6 Примеры программных реализаций RNN.

Регрессионный анализ. Модель RNN сети.

- Рассмотрим регрессионную модель полносвязной сети, которая ранее применялась для одномерного набора данных (X,Y) с добавлением одного рекуррентного слоя.

Layer (type)	Output Shape	Param #
lstm_40 (LSTM)	(None, 20, 128)	66560
flatten_36 (Flatten)	(None, 2560)	0
dense_252 (Dense)	(None, 256)	655616
dense_253 (Dense)	(None, 512)	131584
dense_254 (Dense)	(None, 256)	131328
dense_255 (Dense)	(None, 128)	32896
dense_256 (Dense)	(None, 56)	7224
dense_257 (Dense)	(None, 1)	57
Total params: 1025265 (3.91 MB)		
Trainable params: 1025265 (3.91 MB)		
Non-trainable params: 0 (0.00 Byte)		

5.6 Примеры программных реализаций RNN.

Регрессионный анализ. Модель RNN сети.

❑ Результат обучения с валидацией:

```
m=model.fit(X_train, Y_train, validation_data=(X_val, Y_val), callbacks=callbacks_list,  
            epochs=250, batch_size=16)
```

Epoch 163/250

```
8/8 [=====] - 0s 35ms/step - loss: 4.6717 - root_mean_squared_error: 2.161  
4 - val_loss: 35.1209 - val_root_mean_squared_error: 5.9263 - lr: 1.5625e-05
```

❑ Результат тестирования:

```
test_loss: 34.28813171386719    test_acc: 5.855607032775879  
2/2 [=====] - 0s 7ms/step  
5.8556016965681135  
0.9995930899515488
```